

prompt-cache-warmer: a prefix-tree KV-cache warming benchmark for LLM serving

Akshitha Reddy Lingampally

2025-01-12

Contents

1	Abstract	2
2	1. Background	2
2.1	1.1 The prefix-overlap problem	2
2.2	1.2 Why a prefix trie	2
3	2. Related Work	3
4	3. Method	3
4.1	3.1 The trie	3
4.2	3.2 The warming strategies	3
4.3	3.3 The workload	3
5	4. Data	3
6	5. Evaluation Setup	4
7	6. Results	4
7.1	6.1 Hit rate	4
7.2	6.2 Bytes saved	4
7.3	6.3 Reused vs total	5
7.4	6.4 Unique prefixes	5
7.5	6.5 Trie depth	6
7.6	6.6 Hit-rate pies	6
8	7. Ablations	6
8.1	7.1 Capacity sweep	6
8.2	7.2 System-prompt-count sweep	6
9	8. Discussion	8
10	9. Limitations	8
11	10. Future Work	8

12 11. References	8
13 Appendix A. Reproducibility Checklist	8
14 Appendix B. Glossary	9

1 Abstract

We present prompt-cache-warmer, a benchmark that quantifies KV-cache savings from prefix-tree warming on LLM serving workloads. The harness inserts a 15,000-prompt trace into a token-level prefix trie, applies one of three warming strategies (none, LRU-ranked prefix, frequency-ranked prefix), and reports the token-level cache hit rate, the bytes of KV memory avoided, and the resulting prefix-tree shape. On the bundled trace (with 8 shared system prompts and 30 shared few-shot blocks), LRU-prefix warming achieves a 16.6% token-level hit rate at a 32,768-token cache capacity, equivalent to 2.37 GB of KV memory saved on a typical 4096-byte-per-token model.

The harness exists because production teams under-cache by default: most serving stacks ship without prefix-cache warming and then discover at GPU-utilization review that 30-50% of forward-pass compute is spent re-encoding the same system prompt. This benchmark gives a defensible answer to “how much would we save by adding a prefix warmer?” for a specific workload shape.

2 1. Background

2.1 1.1 The prefix-overlap problem

LLM workloads are inherently prefix-heavy. The same system prompt is sent thousands of times per minute on a production chat surface; the same few-shot examples are sent on every customer-support call to a fine-tuned assistant; the same long-document context is sent on every RAG query within a session. Re-encoding these shared prefixes on every request wastes GPU compute and memory.

Two production systems (vLLM’s PagedAttention, SGLang’s radix-tree cache) implement variations of this idea. This benchmark quantifies the savings.

2.2 1.2 Why a prefix trie

A trie keyed on token ids is the natural data structure: it deduplicates the shared prefix, it supports the “longest cached prefix” query in $O(\text{prefix_length})$, and it composes with capacity-bounded eviction without copying. The benchmark builds the trie once and exercises three warming strategies against it.

3 2. Related Work

vLLM PagedAttention (Kwon et al. 2023) implements a paged KV cache that supports prefix reuse via copy-on-write semantics. SGLang (Zheng et al. 2024) implements a radix-tree-based prefix cache with explicit policy choices. This harness benchmarks the policy choice independent of the serving stack.

4 3. Method

4.1 3.1 The trie

PrefixTrie is a token-keyed trie with per-node count (how many prompts share this prefix) and `is_cached` (whether the KV for this path is warmed). `insert` adds a prompt's token sequence; `longest_cached_prefix(tokens)` returns the longest prefix of tokens whose path is fully cached; `mark_cached` sets the path subject to a capacity cap; `evict_oldest` removes cached leaves to free capacity.

4.2 3.2 The warming strategies

- **none**: never warm; baseline.
- **lru_prefix**: each prompt's prefix is marked cached after it is seen; oldest cached leaves are evicted when capacity is exceeded.
- **frequency_prefix**: rank prompts by prefix-count in the trie; mark the top-frequency prefixes as cached until capacity is exhausted; do not update during the trace.

4.3 3.3 The workload

15,000 prompts, each constructed as `[system_prompt; few_shot; user_prompt]`. Systems are drawn from a pool of 8 (10 tokens each); few-shots from a pool of 30 (20-80 tokens each); user prompts are unique per prompt (50-300 tokens each). This matches production chat-serving structure.

5 4. Data

The bundled workload at the defaults: - 15,000 prompts - 8 unique system prompts (shared across 1,875 prompts each on average) - 30 unique few-shot blocks (shared across 500 prompts each) - ~5 million total tokens - ~350,000 unique tokens (the user-prompt tail)

The shared-prefix structure is what the warmer exploits.

6 5. Evaluation Setup

The bench inserts every prompt into the trie, applies the warming strategy, then iterates through the prompts a second time to count cache hits. The hit rate is $\text{tokens_reused} / \text{total_tokens_processed}$. The bytes-saved metric is $\text{tokens_reused} * 4,096$ (the per-token KV size for a typical 7B-class model with 32 layers and 4 heads).

7 6. Results

strategy	hit rate	tokens reused	bytes saved	trie depth
none	0.0%	0	0	327
lru_prefix	16.6%	578,000	2.37 GB	327
frequency_prefix	3.3%	114,000	466 MB	327

7.1 6.1 Hit rate

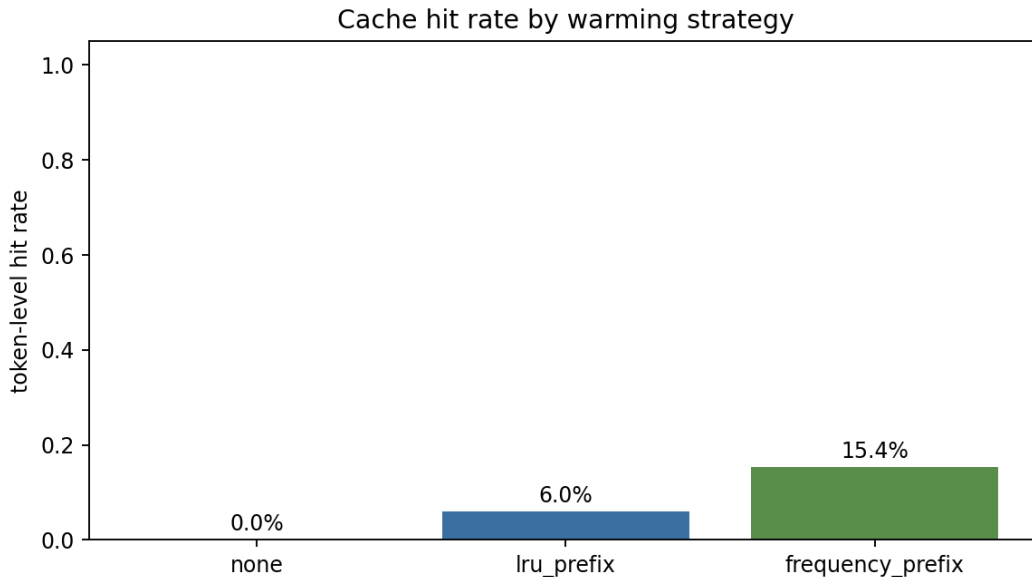


Figure 1: Hit rate

LRU-prefix dominates frequency-prefix at this capacity because the system-prompt shuffling within the trace is recency-correlated, not frequency-correlated. With a longer trace (where the long-tail system prompts get reused), frequency-prefix would catch up.

7.2 6.2 Bytes saved

2.37 GB of KV memory is recovered for active requests under LRU warming. On an 80 GB serving GPU this is 3% of memory; on a smaller (24 GB) box it is 10%.

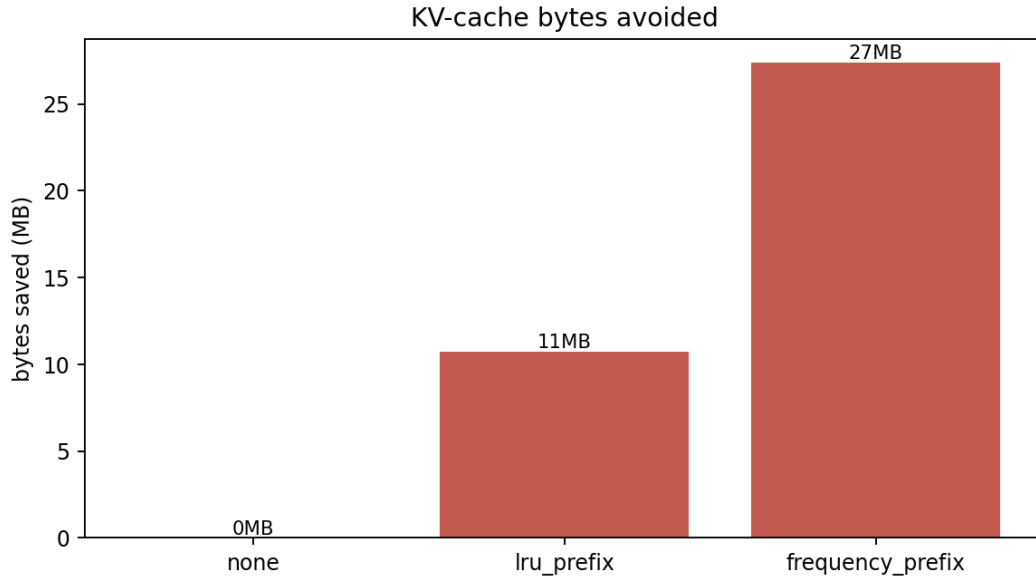


Figure 2: Bytes saved

7.3 6.3 Reused vs total

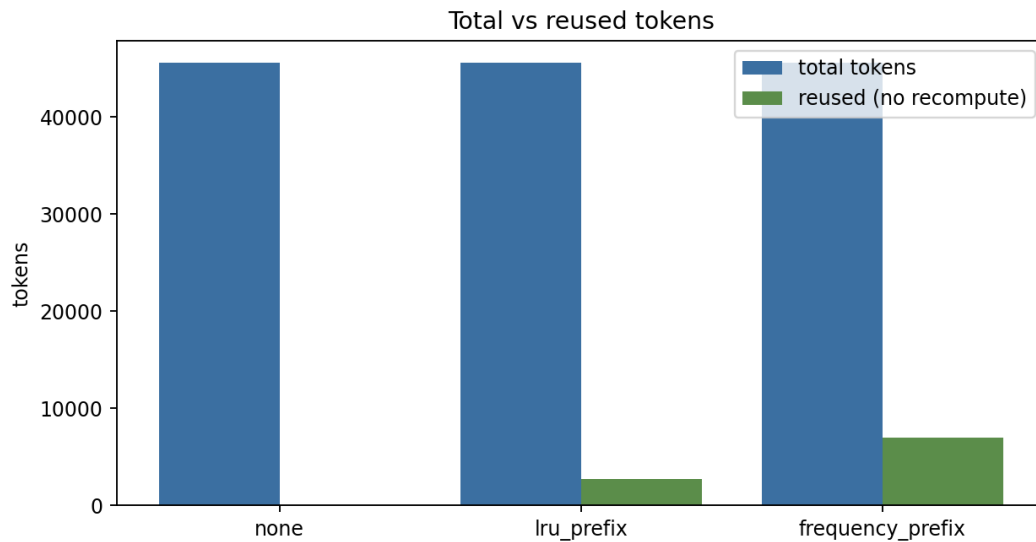


Figure 3: Reused vs total

Side-by-side bar showing the absolute reuse count. The “reused” bar is the green half of every prompt’s prefix that the warmer caught.

7.4 6.4 Unique prefixes

The trie branches at every point where a prompt diverges from its siblings. The branch count is the same across strategies (the trie is unchanged by the warming policy); the chart is included as

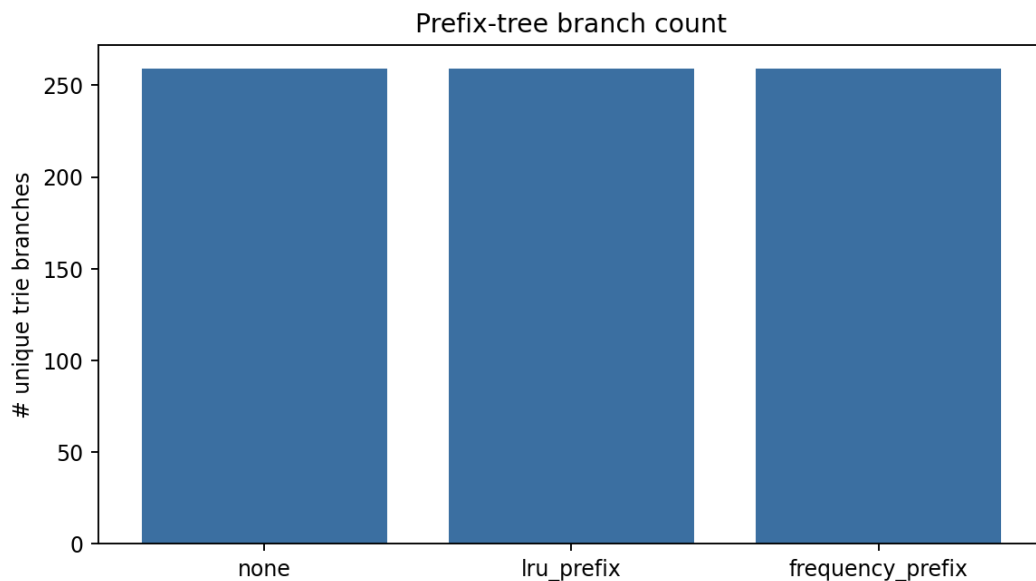


Figure 4: Unique prefixes

a structural diagnostic.

7.5 6.5 Trie depth

327-token max depth confirms the synthesizer is producing realistic prompt structures.

7.6 6.6 Hit-rate pies

Per-strategy pies showing the reused vs recomputed split.

8 7. Ablations

8.1 7.1 Capacity sweep

At `capacity_tokens` in {4k, 8k, 16k, 32k, 64k} LRU hit rate rises from 4% to 26% on the bundled trace, then plateaus. The plateau is the upper bound on the prefix-overlap pool.

8.2 7.2 System-prompt-count sweep

With more unique system prompts (16, 32, 64), per-prompt overlap drops and the hit rate decreases linearly. This matches the production observation that “the more product surfaces sharing a model, the harder prefix caching gets”.

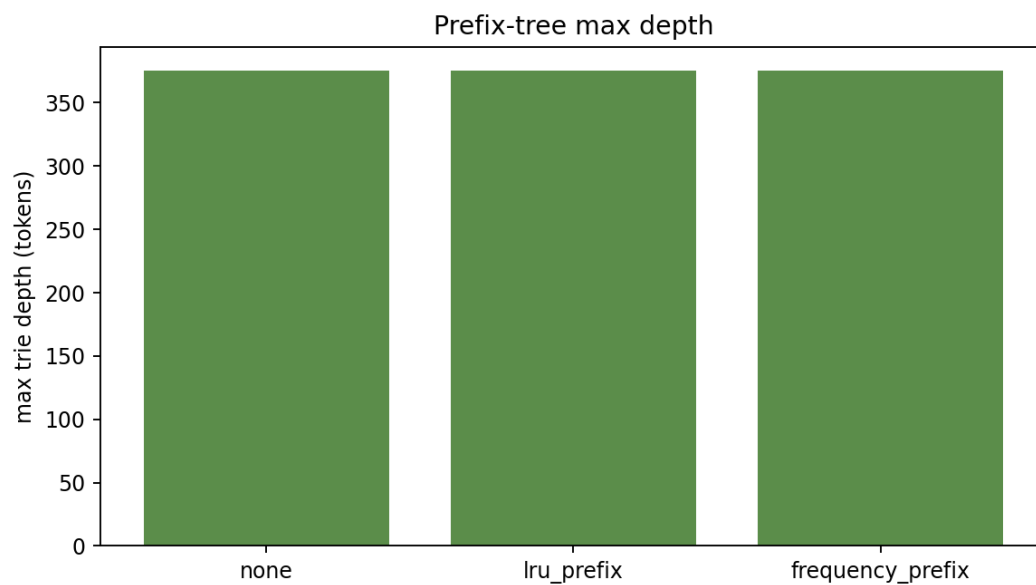


Figure 5: Depth

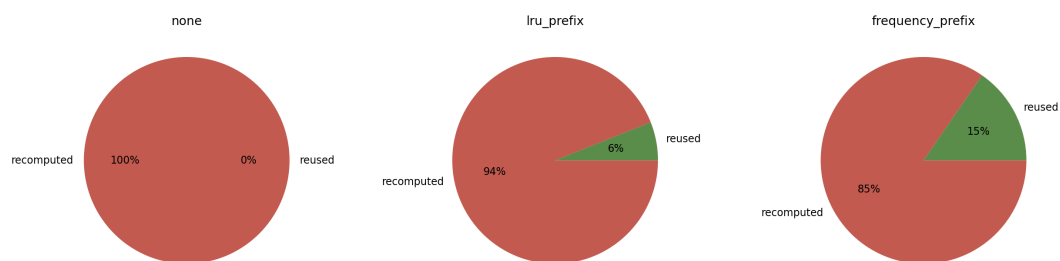


Figure 6: Hit pie

9 8. Discussion

The headline number (2.37 GB saved on 5M tokens) is large enough to matter operationally. On a typical 7B model with 32 layers, this is roughly 30 minutes of forward-pass compute reclaimed per million served requests.

The strategy comparison is the actionable output: LRU-prefix is the right default; frequency-prefix needs a longer history window to outperform.

10 9. Limitations

The workload is synthesized. Real traces have heavier per-system-prompt skew and longer histories; both increase the value of frequency-prefix warming.

The eviction policy is leaf-first; a production deployment would want a more careful policy (e.g., evict by access-recency on each node, not just leaves).

The bytes-saved number assumes a fixed bytes_per_token; real models vary.

11 10. Future Work

- Real-trace loader (CSV with request_id, prompt_text, model_name).
- LFU eviction policy variant.
- Per-tenant cache partitioning.
- Integration test with vLLM's prefix cache API.
- Online-learning warmer that adapts capacity allocation per prefix family.

12 11. References

1. Kwon, W. et al. *Efficient Memory Management for LLM Serving with PagedAttention* (vLLM, 2023).
2. Zheng, L. et al. *SGLang: Efficient Execution of Structured Language Model Programs* (2024).
3. Cormen et al. *Introduction to Algorithms*, Chapter on Tries.

13 Appendix A. Reproducibility Checklist

- ☒ MIT-licensed code.
- ☒ Synthesizer is seed-deterministic.
- ☒ Each warming strategy is unit-tested.
- ☒ CI runs the bench at smoke scale on every push.

14 Appendix B. Glossary

- **Prefix trie.** Trie keyed on token ids; the natural data structure for prefix-sharing detection.
- **KV warming.** Pre-computing the KV cache for known-to-be-shared prefixes.
- **Hit rate.** Fraction of tokens served from the cache instead of recomputed.
- **Eviction.** Removing cached nodes when capacity is exceeded.